Лабораторная работа №2
Продвинутые методы безусловной оптимизации

Егор Гуров Александр Громоздин Николай Гавришок

2026

Титульная информация

- **Пакет оракулов и датасетов:** №2.
- **Классификация:** Logistic Regression + L_2.
- **Регрессия:** Poisson Regression + L_2.
- **Индивидуальный трек:** трек 4, Cautious L-BFGS и отказ от Вульфа.
- **Невыпуклая 2D-функция:** функция Била.

Распределение работы

Егор Гуров Аналитический вывод формул для Logistic Regression, Poisson Regression и произведения гессиана на вектор; проверка корректности оракулов и разностных формул; математическая часть отчета; описание экспериментов 2.2 и 2.3; участие в анализе трека 4.

Александр Громоздин Реализация `LineSearchTool`, `linear_conjugate_gradients`, `nonlinear_conjugate_gradients`, `hessian_free_newton`; интеграция ноутбука и основных экспериментов; анализ поведения методов на *phishing* и *real-sim*.

Николай Гавришок Реализация `hess_vec`, `hess_vec_finite_diff`, `lbfgs`, `cautious_lbfgs`; запуск экспериментов 2.4–2.6 и трека 4; подготовка графиков и таблиц; верстка итогового отчета и презентации.

Используемая среда

Эксперименты запускались на MacBook Pro с процессором Apple M4 и 16 GB unified memory. Использовался Python 3.14 и библиотеки `numpy`, `scipy`, `matplotlib`, `scikit-learn`, `jupyter`. Для базовых ML-экспериментов использовалась начальная точка $x_0 = 0$, коэффициент регуляризации $\lambda = 1/m$. В эксперименте 2.6 данные разделялись на train/test в пропорции 80/20. Для трека 4 использовалась случайная начальная точка $x_0 \sim \mathcal{N}(0, 10)$.

1 Реализованные методы и оракулы

В работе реализованы линейный метод сопряженных градиентов, нелинейный метод сопряженных градиентов с формулой Полака–Рибьера и рестартом, усеченный метод Ньютона, L-BFGS, Cautious L-BFGS, градиентный спуск и полный метод Ньютона. Для выбора длины шага используется общий класс линейного поиска: поддерживаются постоянный шаг, бэктрекинг Армихо и поиск по условиям Вульфа.

Для ML-задач реализованы оракулы Logistic Regression + L_2 и Poisson Regression + L_2. В обоих оракулах метод `hess_vec` считает произведение гессиана на вектор без явного формирования гессиана. Это важно для усеченного метода Ньютона и для работы с разреженными датасетами.

2 Проверка произведения гессиана на вектор

Постановка

Перед запуском методов второго порядка была проверена корректность `hess_vec`. Для этого на небольшой модельной выборке сравнивалось аналитическое произведение $\nabla^2 f(x)v$ и конечная разностная аппроксимация. Проверка выполнялась для плотной и разреженной матрицы признаков, а также для обоих ML-оракулов.

Результаты

Тип матрицы	Оракул	Минимальная ошибка	Максимальная ошибка
Dense	LogisticL2	$8.46 \cdot 10^{-6}$	$3.61 \cdot 10^{-5}$
Dense	PoissonL2	$7.03 \cdot 10^{-5}$	$1.36 \cdot 10^{-3}$
Sparse	LogisticL2	$7.62 \cdot 10^{-6}$	$3.35 \cdot 10^{-5}$
Sparse	PoissonL2	$3.52 \cdot 10^{-5}$	$4.91 \cdot 10^{-2}$

Таблица 1: Проверка `hess_vec`: евклидова норма разности между аналитическим и разностным произведением гессиана на вектор.

Выводы

Для Logistic Regression ошибка стабильно находится на уровне 10^{-5} как для плотной, так и для разреженной матрицы. Для Poisson Regression ошибка иногда больше из-за экспоненты и чувствительности конечных разностей, но порядок ошибок остается приемлемым для численной проверки. Таким образом, реализация `hess_vec` согласуется с разностной аппроксимацией и может использоваться во внутреннем цикле усеченного Ньютона.

3 Эксперимент 2.2. Метод сопряженных градиентов против градиентного спуска

Постановка

Исследовалась квадратичная задача

$$f(x) = \frac{1}{2}x^T Ax - b^T x,$$

где A – диагональная положительно определенная матрица. Собственные значения A выбирались геометрической сеткой от 1 до κ, поэтому число обусловленности равно κ. Сравнивались линейный CG и градиентный спуск с линейным поиском Вульфа. Рассматривались размерности $n \in \{10, 100, 300\}$ и $\kappa \in \{1, 10, 10^2, 10^3, 10^4\}$.

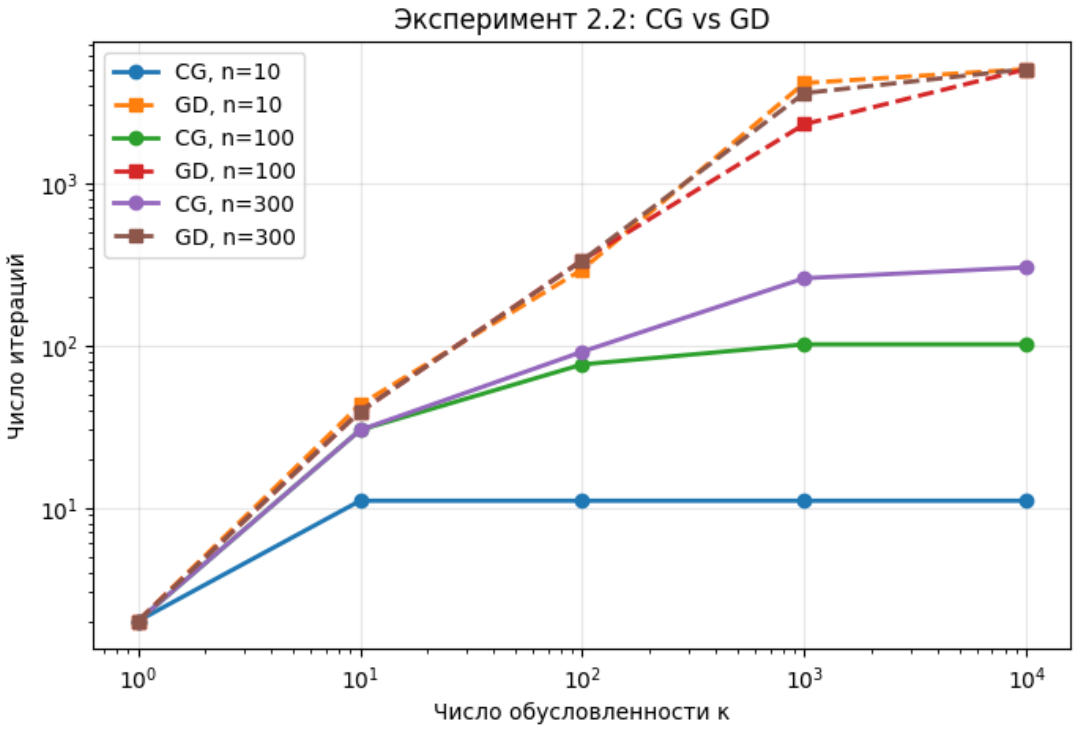


Рис. 1: Эксперимент 2.2: число итераций CG и GD в зависимости от числа обусловленности и размерности.

n	Метод	$\kappa = 1$	$\kappa = 10$	$\kappa = 10^2$	$\kappa = 10^3$
10	CG	1	10	10	10
10	GD	1	42	374	4103
100	CG	1	29	75	100
100	GD	1	38	313	2273
300	CG	1	29	90	257
300	GD	1	38	290	3271

Таблица 2: Количество итераций до достижения заданной точности. При $\kappa = 10^4$ градиентный спуск достигает лимита 5000 итераций для всех рассмотренных размерностей, а CG укладывается не более чем в n итераций.

Выводы

Метод сопряженных градиентов существенно лучше градиентного спуска на плохо обусловленных квадратичных задачах. Для SPD-квадратичной функции линейный CG в точной арифметике находит решение не более чем за n шагов, что хорошо видно на графике: даже при $\kappa = 10^4$ число итераций ограничено размерностью. У градиентного спуска рост числа итераций гораздо сильнее зависит от κ, поэтому при плохой обусловленности он быстро становится непрактичным.

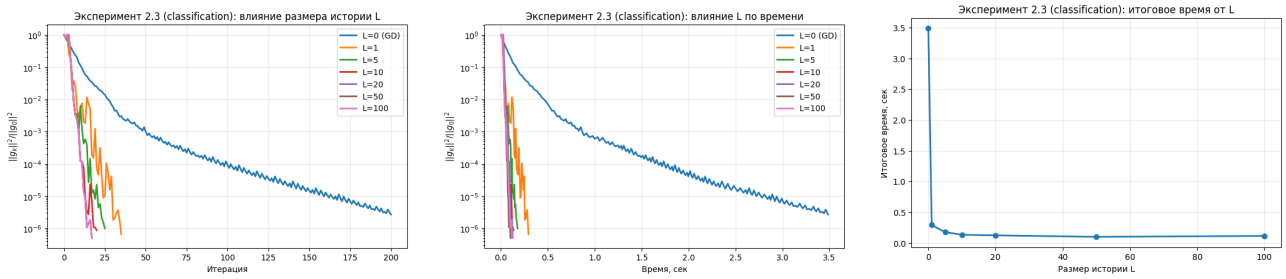
4 Эксперимент 2.3. Выбор размера истории в L-BFGS

Постановка

Исследовалось влияние размера истории L в L-BFGS. Были рассмотрены значения $L = 0, 1, 5, 10, 20, 50, 100$. Случай $L = 0$ соответствует обычному градиентному спуску с линейным поиском Вульфа. Эксперимент проводился на задачах классификации и регрессии из пакета №2, а также на плохо обусловленной квадратичной функции с $\kappa = 1000$.

Память метода составляет порядка $2Ln$ чисел, поскольку хранятся L пар векторов (s_k, y_k). Стоимость вычисления направления через двухцикловую рекурсию равна $O(Ln)$ без учета стоимости оракула. Поэтому слишком большое L может уменьшать число итераций, но замедлять каждую итерацию.

Результаты на задаче классификации



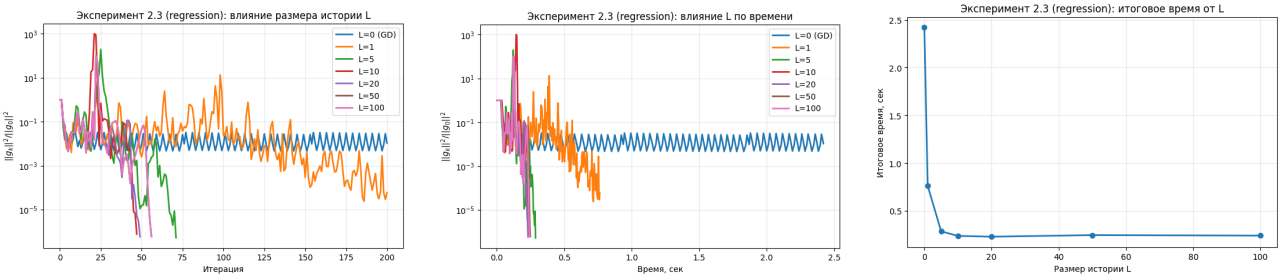
(a) по итерациям (b) по времени (c) итоговое время

Рис. 2: Эксперимент 2.3: влияние размера истории L на Logistic Regression.

L	Итерации	Время, с	Финальный относительный квадрат нормы градиента
0	200	3.486555	$2.679916 \cdot 10^{-6}$
1	35	0.293081	$6.531834 \cdot 10^{-7}$
5	25	0.177948	$9.918553 \cdot 10^{-7}$
10	20	0.134737	$8.759874 \cdot 10^{-7}$
20	17	0.125572	$5.059502 \cdot 10^{-7}$
50	17	0.101598	$5.059502 \cdot 10^{-7}$
100	17	0.116461	$5.059502 \cdot 10^{-7}$

Таблица 3: Эксперимент 2.3: сводка для классификации.

Результаты на задаче регрессии



(a) по итерациям (b) по времени (c) итоговое время

Рис. 3: Эксперимент 2.3: влияние размера истории L на Poisson Regression.

L	Итерации	Время, с	Финальный относительный квадрат нормы градиента
0	200	2.420565	$1.061185 \cdot 10^{-2}$
1	200	0.763909	$5.917132 \cdot 10^{-5}$
5	71	0.287749	$5.238313 \cdot 10^{-7}$
10	47	0.240076	$7.653048 \cdot 10^{-7}$
20	49	0.231979	$5.808728 \cdot 10^{-7}$
50	56	0.248421	$5.970100 \cdot 10^{-7}$
100	56	0.243428	$6.019412 \cdot 10^{-7}$

Таблица 4: Эксперимент 2.3: сводка для регрессии.

Плохо обусловленная квадратичная функция

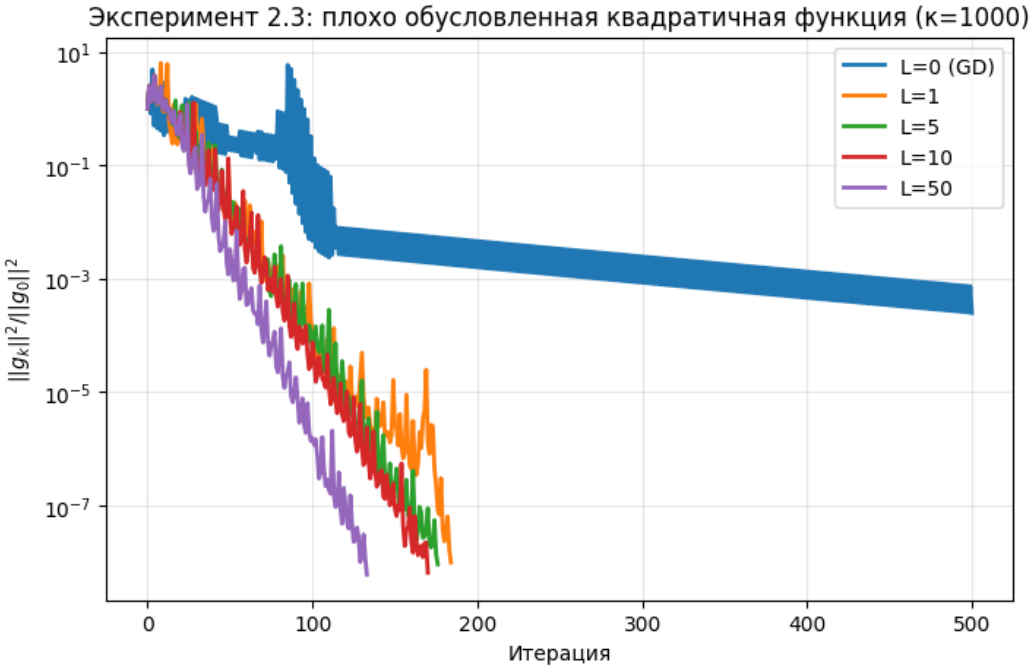


Рис. 4: Эксперимент 2.3: L-BFGS на плохо обусловленной квадратичной функции с $\kappa = 1000$.

Выводы

Уже минимальная история резко ускоряет метод по сравнению с $L = 0$. На классификации переход от $L = 0$ к $L = 1$ сокращает время с 3.49 с до 0.29 с, а число итераций – с 200 до 35. На регрессии $L = 1$ еще не дает полной сходимости за 200 итераций, но уже значительно уменьшает финальную норму градиента.

Дальнейшее увеличение L дает плато. На классификации после $L \approx 20$ число итераций уже не уменьшается: для $L = 20, 50, 100$ получается 17 итераций. На регрессии оптимальный баланс по времени находится около $L = 20$: при $L = 50$ и $L = 100$ число итераций становится больше, а стоимость двухцикловой рекурсии возрастает. Это подтверждает, что слишком большая история не всегда полезна: метод тратит больше времени на обработку старых пар, которые уже слабо улучшают аппроксимацию кривизны.

На плохо обусловленной квадратичной функции большая история полезнее, чем на хорошо обусловленных задачах. При сильной коррелированности признаков спектр гессиана шире, и методу нужно хранить больше информации о кривизне по разным направлениям. Поэтому для плохо обусловленных задач оптимальное L обычно больше, чем для хорошо масштабированных данных.

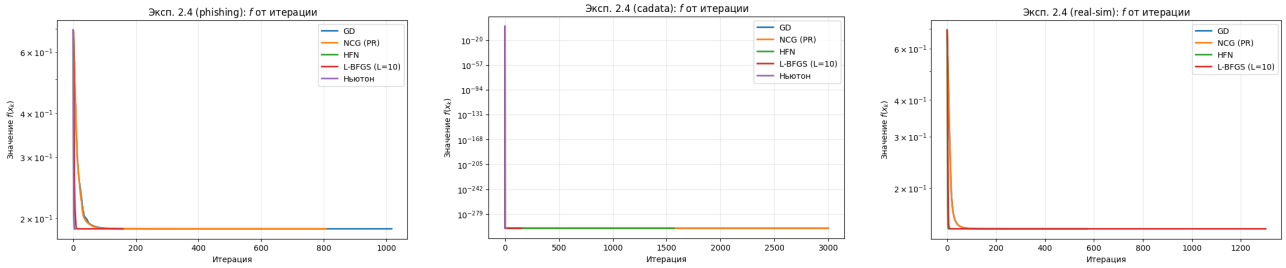
5 Эксперимент 2.4. Сравнение методов на реальных ML-задачах

Постановка

Сравнивались GD, NCG, HFN, L-BFGS и полный метод Ньютона. Для L-BFGS использовался размер истории $L = 10$. Для HFN внутренний CG останавливался по адаптивному критерию с форсирующей последовательностью. Для всех методов использовался относительный критерий остановки по норме градиента $\|\nabla f(x_k)\| \leq 10^{-6}\|\nabla f(x_0)\|$.

Использовались датасеты *phishing*, *cadata* и *real-sim*. Для *real-sim* полный метод Ньютона не запускался, поскольку явный гессиан при большой размерности признаков требует слишком много памяти.

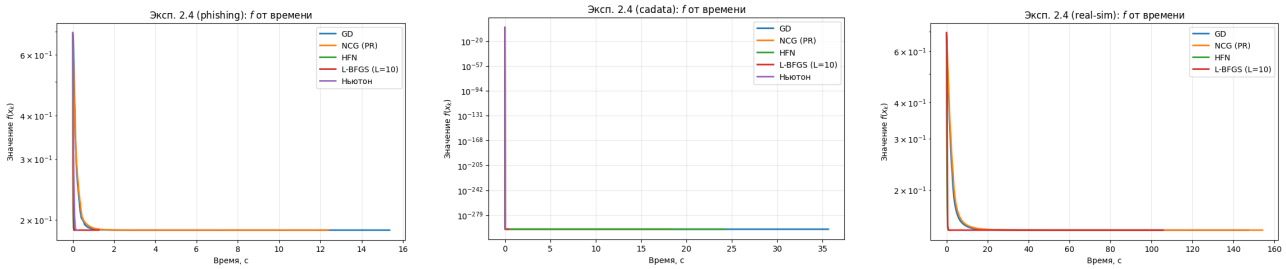
Значение функции от номера итерации



(a) phishing (b) cadata (c) real-sim

Рис. 5: Эксперимент 2.4: значение функции от номера итерации.

Значение функции от реального времени



(a) phishing (b) cadata (c) real-sim

Рис. 6: Эксперимент 2.4: значение функции от времени.

Относительный квадрат нормы градиента от времени

(a) phishing (b) cadata (c) real-sim

Рис. 7: Эксперимент 2.4: относительный квадрат нормы градиента от времени.

Датасет	Метод	Статус	Итерации	Время, с
phishing	GD	success	1017	15.356451
phishing	NCG (PR)	success	807	12.377264
phishing	HFN	success	6	0.096513
phishing	L-BFGS	success	159	1.261580
phishing	Ньютон	success	6	0.254475
cadata	GD	exceeded	3000	35.624215
cadata	NCG (PR)	exceeded	3000	18.225778
cadata	HFN	success	1567	24.176706
cadata	L-BFGS	success	153	0.367462
cadata	Ньютон	success	8	0.072186
real-sim	GD	success	573	147.242894
real-sim	NCG (PR)	success	573	154.213862
real-sim	HFN	success	7	1.821343
real-sim	L-BFGS	success	1302	105.656599

Таблица 5: Эксперимент 2.4: сводка по числу итераций и времени.

Выводы

По числу внешних итераций почти всегда лидируют методы второго порядка: полный Ньютон и HFN. На *phishing* HFN и полный Ньютон сходятся за 6 итераций, но HFN быстрее по времени, потому что не строит и не факторизует полный гессиан. На *cadata* полный Ньютон оказывается самым быстрым, так как размерность позволяет дешево работать с плотным гессианом.

На *real-sim* полный Ньютон не применим из-за размера гессиана. В этой широкой разреженной задаче HFN становится лучшим по времени: он использует только произведения гессиана на вектор и сходится за 7 внешних итераций. L-BFGS здесь делает много итераций, но остается существенно быстрее GD и NCG. Обычный градиентный спуск и NCG на реальных задачах проигрывают продвинутым методам, особенно по времени достижения высокой точности.

6 Эксперимент 2.5. Микропрофилирование вычислительных затрат

Постановка

Для NCG, HFN и L-BFGS замерялось среднее распределение времени одной итерации между тремя компонентами: вычисления оракула, линейная алгебра алгоритма и линейный поиск.

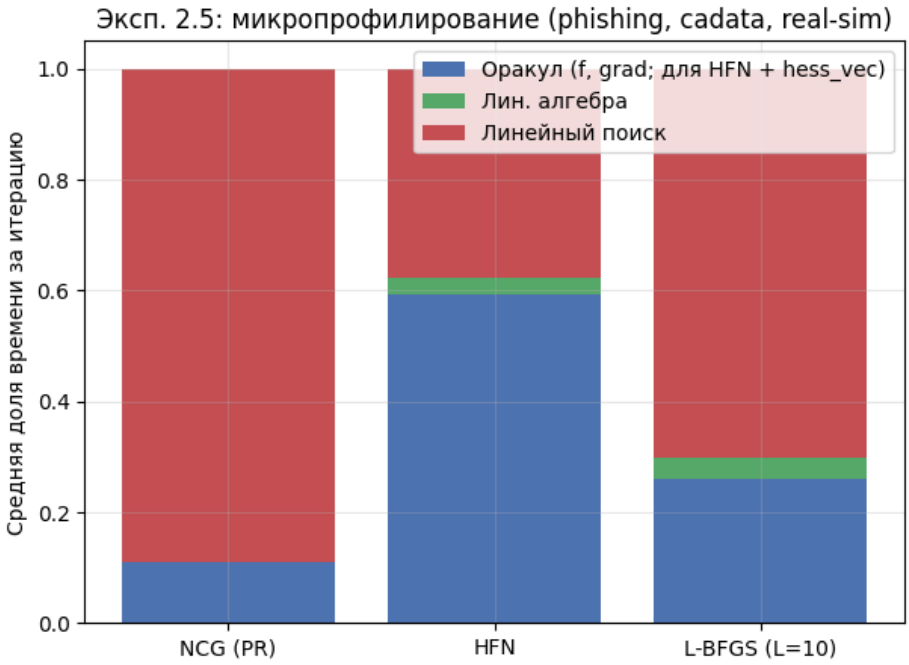


Рис. 8: Эксперимент 2.5: средние доли времени по компонентам.

Метод	Оракул	Линейная алгебра	Линейный поиск
NCG (PR)	0.109	0.001	0.890
HFN	0.592	0.031	0.377
L-BFGS	0.261	0.037	0.702

Таблица 6: Эксперимент 2.5: доли времени, усредненные по запускам.

Выводы

Самую большую долю времени на линейный поиск тратит NCG: около 89% времени. Это объясняется тем, что направление NCG не так хорошо учитывает кривизну, поэтому подбор шага по Вульфу становится дорогим. У L-BFGS линейный поиск также дорогой и занимает около 70% времени, но направление обычно намного качественнее, чем у градиентных методов.

У HFN основная стоимость приходится на оракул: около 59% времени. Это связано с тем, что во внутреннем цикле CG многократно вызывается `hess_vec`. Поэтому на широких задачах $n \gg m$ HFN может проигрывать L-BFGS по времени, если внутренний CG требует много произведений гесси-

ана на вектор. В наших запусках на *real-sim* HFN все равно выиграл, потому что число внешних итераций оказалось очень маленьким.

7 Эксперимент 2.6. Оптимизационная точность и качество предсказания

Постановка

Цель эксперимента – проверить, совпадает ли оптимизационная сходимость с улучшением качества на тестовой выборке. Для этого L-BFGS запускался с высокой точностью, а на каждой итерации фиксировались train loss, относительный квадрат нормы градиента и тестовая метрика. Для классификации использовалась Accuracy, для регрессии – MSE. Дополнительно сравнивались три значения регуляризации: $0.1/m$, $1/m$ и $10/m$.

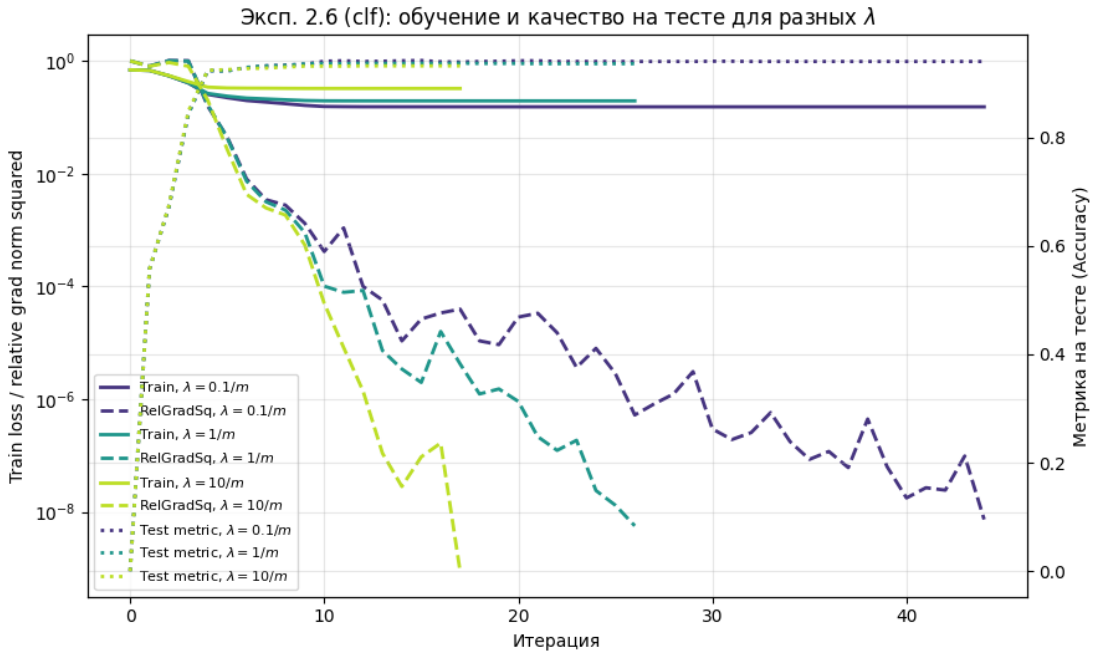


Рис. 9: Эксперимент 2.6: классификация на phishing. Левая ось – train loss и относительный квадрат нормы градиента в логарифмической шкале, правая ось – Accuracy на тесте.

Рис. 10: Эксперимент 2.6: регрессия на cadata. Левая ось – train loss и относительный квадрат нормы градиента, правая ось – MSE на тесте.

Задача и λ	Лучшая тестовая метрика	Итерация насыщения	Бесполезное время, %
Classification, $0.1/m$	0.941655	8	80.93
Classification, $1/m$	0.936680	6	77.02
Classification, $10/m$	0.931705	4	75.34
Regression, $0.1/m$	$4.937354 \cdot 10^{10}$	0	99.99
Regression, $1/m$	$4.937354 \cdot 10^{10}$	0	99.99
Regression, $10/m$	$4.937354 \cdot 10^{10}$	0	99.99

Таблица 7: Эксперимент 2.6: насыщение тестовой метрики. Для классификации больше – лучше, для регрессии меньше – лучше.

Выводы

На классификации качество на тесте выходит на плато намного раньше, чем оптимизационный критерий. При стандартном $\lambda = 1/m$ лучшая Accuracy 0.93668 достигается уже около 6-й итерации, после чего train loss и норма градиента продолжают уменьшаться, но тестовая метрика почти не меняется. Около 77% общего времени уходит на оптимизацию, которая уже почти не улучшает качество на тесте. Для практических задач в такой ситуации разумнее использовать более мягкую точность или ранний останов по валидационной метрике.

При уменьшении регуляризации до $0.1/m$ лучшая Accuracy становится выше, но насыщение наступает позже, а доля поздней оптимизации увеличивается. При увеличении регуляризации до $10/m$ качество ниже, но насыщение наступает раньше. Это согласуется с обычной картиной: более слабая регуляризация позволяет лучше подстроиться под train, но увеличивает риск разрыва между train loss и тестовой метрикой.

На регрессии с Poisson-моделью выбранная постановка оказалась плохо согласована с данными *cadata*: тестовая MSE не улучшалась содержатель-

но, а лучшая метрика достигалась уже в начале. Поэтому здесь особенно видно, что уменьшение оптимизационной ошибки само по себе не гарантирует улучшения предсказательного качества.

8 Исследовательский трек 4. Cautious L-BFGS и отказ от Вульфа

Постановка

В классическом L-BFGS положительная определенность аппроксимации обратного гессиана поддерживается за счет условия кривизны $\langle s_k, y_k \rangle > 0$, которое обычно обеспечивается линейным поиском по Вульфу. В треке 4 проверяется идея Cautious L-BFGS: заменить дорогой Wolfe search на более дешевый Armijo backtracking, но добавлять пару (s_k, y_k) в историю только при выполнении условия

$$\frac{\langle s_k, y_k \rangle}{\|s_k\|^2} > \varepsilon \|\nabla f(x_k)\|^\alpha, \qquad \varepsilon = 10^{-4}, \quad \alpha = 1.$$

Если условие не выполнено, шаг принимается, но история L-BFGS не обновляется. Дополнительно был проведен стресс-тест с $\varepsilon = 10^{-1}$, чтобы явно увидеть механизм отбрасывания пар.

Результаты на phishing

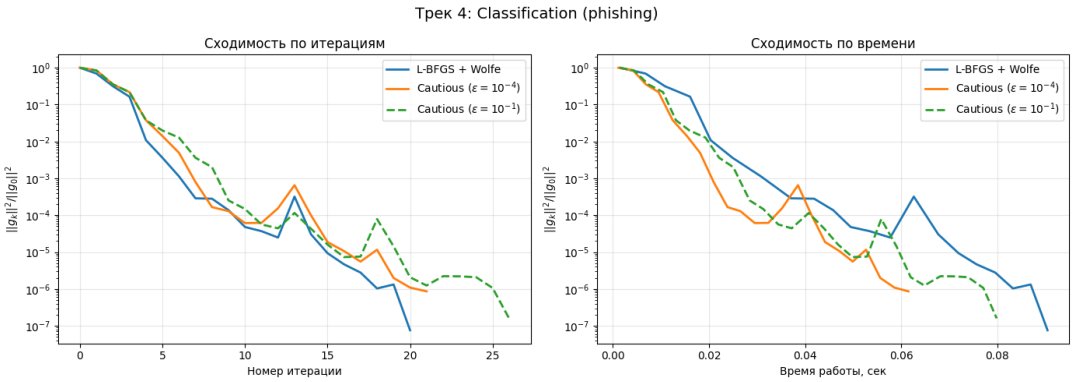


Рис. 11: Трек 4: сравнение L-BFGS + Wolfe и Cautious L-BFGS + Armijo на phishing.

Метод	Статус	Средние вызовы оракула	Пропущено пар	Время, с
L-BFGS + Wolfe	success	6.35	0	0.0905
Cautious, $\varepsilon = 10^{-4}$	success	5.10	0	0.0615
Cautious, $\varepsilon = 10^{-1}$	success	5.12	4	0.0799

Таблица 8: Трек 4: результаты на phishing.

Стресс-кейс на cadata

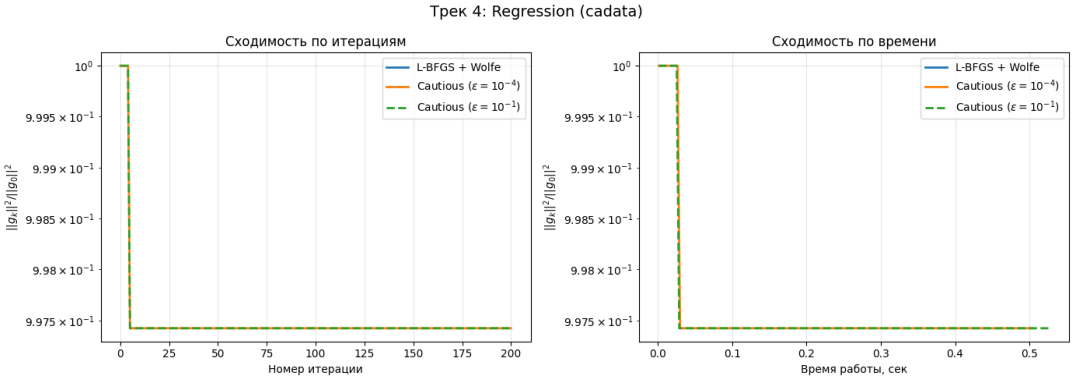


Рис. 12: Трек 4: запуск из случайной точки на cadata.

Метод	Статус	Средние вызовы оракула	Пропущено пар	Время, с
L-BFGS + Wolfe	computational error	16.00	0	0.0012
Cautious, $\varepsilon = 10^{-4}$	iterations exceeded	5.20	199	0.5045
Cautious, $\varepsilon = 10^{-1}$	iterations exceeded	5.20	199	0.5270

Таблица 9: Трек 4: стресс-кейс на cadata.

Выводы

На *phishing* Cautious L-BFGS с Armijo сохраняет устойчивость без Вульфа и выигрывает по времени. При рекомендованном $\varepsilon = 10^{-4}$ пары не отбрасываются, то есть данные и начальная точка не приводят к нарушению условия кривизны. При искусственном ужесточении до $\varepsilon = 10^{-1}$ алгоритм отбрасывает 4 пары, но все равно сходится. Это показывает, что механизм осторожного обновления действительно работает: сомнительные пары не попадают в историю, а метод продолжает двигаться за счет уже накопленной аппроксимации или градиентного направления.

По среднему числу вызовов оракула Armijo дешевле Wolfe. На *phishing* классический L-BFGS требует 6.35 вызовов оракула на итерацию, тогда как cautious-вариант требует около 5.10 вызовов. Поэтому даже при менее точном поиске шага cautious-вариант оказался быстрее: 0.0615 с против 0.0905 с.

На *cadata* случайная начальная точка является жестким стресс-кейсом для Poisson Regression: Wolfe search быстро ломается, а cautious-метод почти всегда отбрасывает пары и не успевает сойтись за 200 итераций. Этот результат не противоречит идее метода: осторожное правило защищает историю от плохих пар, но если почти все пары отвергаются, L-BFGS фактически теряет квазиньютоновскую информацию и приближается к градиентному методу с Armijo.

Итоговый вывод

Все обязательные алгоритмы и эксперименты лабораторной работы реализованы. Проверка `hess_vec` подтверждает корректность безгессианных про-

изведений. Линейный CG ожидаемо превосходит GD на плохо обусловленных квадратичных задачах. L-BFGS показывает хороший баланс между стоимостью итерации и скоростью сходимости, причем оптимальный размер истории обычно умеренный. HFN делает меньше всего внешних итераций и особенно полезен там, где полный гессиан хранить невозможно. Эксперимент 2.6 показывает, что высокая оптимизационная точность не всегда улучшает качество на тесте. В треке 4 Cautious L-BFGS подтверждает, что отказ от Вульфа может дать выигрыш по времени, если условие осторожного обновления не слишком часто блокирует историю.